

Microservices scaling demo

github: <https://github.com/pesekt1/microservices-scaling>

Contents

System Overview	4
Introduction.....	4
System architecture – without the monitoring and logging services and autoscaling.....	4
Tech stack.....	4
Microservices	5
Bank microservice.....	5
Transactions Receiver microservice	6
Fraud Detection microservice	6
Transaction Processing microservice	6
Message processing speed simulation	6
Dependencies.....	7
Messages	8
Logger as a dependency	8
RabbitMQ.....	12
RabbitMQ Management GUI	14
RabbitMQ Exporter	14
Docker Desktop.....	15

Docker compose	16
Waiting for other services	17
Docker Compose Networks	18
Scaling.....	19
Mounted volumes	20
Kubernetes deployment	20
MySQL in Kubernetes	23
RabbitMQ in Kubernetes.....	24
RabbitMQ Exporter in Kubernetes.....	24
Port forwarding	25
Scaling with Kubernetes	26
Manual scaling	26
Autoscaling	28
Final solution architecture	34
Rolling updates.....	35
Start Kubernetes deployments with shell script	35
Lens	36
Logging	38
Grafana Loki	39
Promtail	41
Monitoring	43

Prometheus	43
Metrics server	46
Grafana	46
Loki data source	48
Prometheus data source	50
Mysql data source	51
CI/CD Pipelines	52
Testing	53
Unit tests	54
Integration tests	54
System tests	55
End-to-end tests	55
Stress tests	55
Load Balancing	55
Using Message Broker	55
HTTP Load Balancing	55
Communication	56
REST APIs	56
Message Broker	56
Postman - sending messages	56
Debugging Grafana Loki in Kubernetes cluster	57

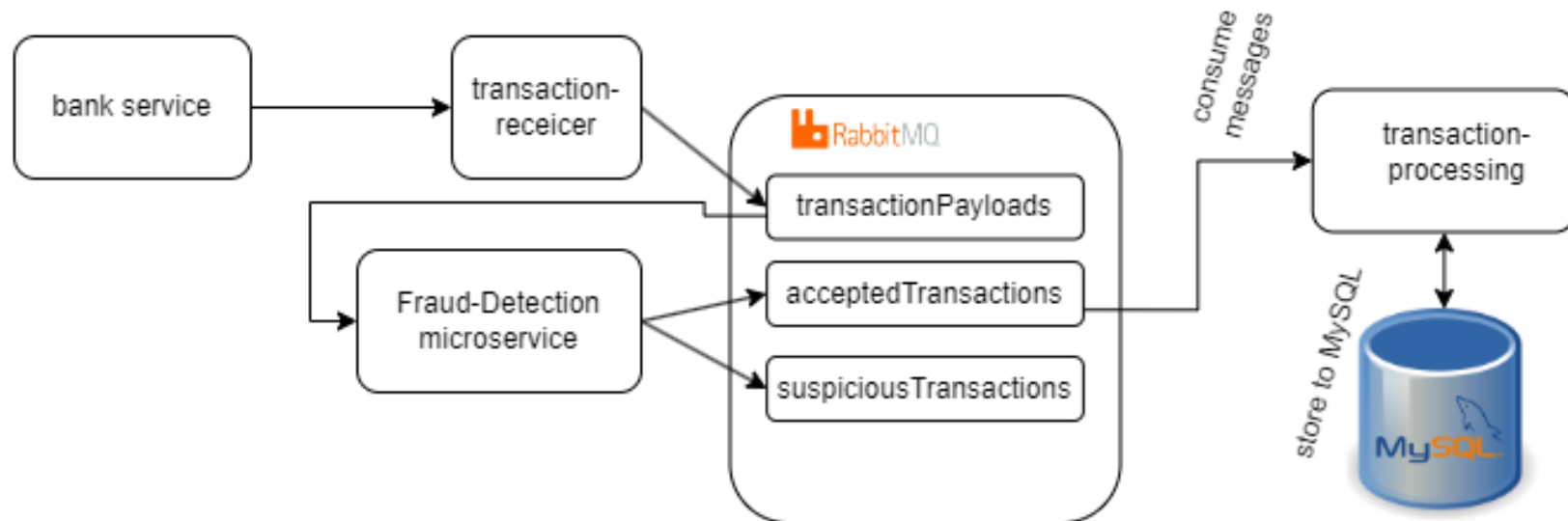
System Overview

Introduction

Demo of a large system using microservice architecture

- Development environment: docker-compose
- Production environment (simulation): Deployment in Local Kubernetes Cluster in Docker Desktop

System architecture - without the monitoring and logging services and autoscaling



Tech stack

- programming
 - Typescript
 - Node.js

- Express.js
- services
 - MySQL
 - RabbitMQ
 - RabbitMQ-Exporter
 - Grafana
 - Loki
 - Promtail
 - Prometheus
 - KEDA
- Deployment
 - Docker & Docker-compose
 - Kubernetes

Microservices

Bank microservice

Simulates the outside service (that is why it is not part of the Kubernetes cluster deployment).

Sends periodically a message (POST HTTPS request with a JSON object) to the Transactions Receiver microservice which is the entry API of our system.

We just use a faker library to send random messages. It is just a simple json object with few properties.

The speed is controlled by a environment variable MESSAGE_SPEED (needs to be provided also in docker-compose and kubernetes deployment manifest).

We also simulate message speed fluctuation (hardcoded in the code).

Transactions Receiver microservice

API that receives the transaction payload from the bank microservice and publishes it to RabbitMQ transactionPayloads queue for further processing.

Fraud Detection microservice

Consumes from transactionPayloads queue and simulates fraud detection process.

- If a message is approved it is published to acceptedTransactions queue.
- If a message is suspicious it is published to suspiciousTransactions queue.

Transaction Processing microservice

Consumes the messages from acceptedTransactions queue and simulate a processing time for each message. To simulate processing of the message, we add a timestamp – processed_at to the object and save it in mysql database.

The speed is controlled by a environment variable MESSAGE_PROCESSING_SPEED (needs to be provided also in docker-compose and kubernetes deployment manifest).

If too many messages are arriving to the queue, the consumer microservice is not able to process them fast enough and they accumulate.

To solve this problem, we need to scale this microservice – explained in Kubernetes part where we implemented autoscaling.

NOTE: You cannot have dropping and recreating the schema on every start:

```
await sequelize.sync({ force: true })
```

The problem is that when scaling, this will cause problems to the other replicas because it will delete the table and recreate it, so the other replicas will fail because they cannot find the table.

Message processing speed simulation

Use an environment variable defining the delay between consuming of 2 messages:

```
const MESSAGE_PROCESSING_SPEED = parseInt(
```

```
process.env.MESSAGE_PROCESSING_SPEED as string
```

Create a delay function:

```
const delay = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
```

For delay simulation to work, we need to prefetch one message at a time before consuming it, then we apply the delay, and we also need to enable manual acknowledgement:

```
export async function consumeMessages() {  
  try {  
    // Limit the number of unacknowledged messages to 1, otherwise the delay will not work  
    channel.prefetch(1);  
  
    channel.consume(  
      INPUT_QUEUE_NAME,  
      async (msg) => {  
        if (msg) {  
          const data = JSON.parse(msg.content.toString());  

```

...

```
    // Simulate processing delay  
    await delay(MESSAGE_PROCESSING_SPEED);
```

...

```
    channel.ack(msg);  
  }  
},  
{ noAck: false } //Ensure manual acknowledgment is enabled otherwise the delay will not work
```

Dependencies

In microservice architecture it is important to decouple microservices from each other. The typical antipattern is to create a common source code for all microservices in a monorepo. It works but it makes the source code coupled. It is better to create an online

package which all microservices can import and use as a dependency. Like this, there is no need to have all the source code in a single monorepo. There is no coupling. We can easily host each microservice separately in its own repository.

Examples:

- npm package (JS / TS)
- pypi (Python)
- dll / nuget package (C#) via NuGet repository
- Jar package via Maven repository

Messages

Message objects are typical dependencies. The producer microservice defines the data type of the message. Consumers can then import a dependency for example in form of a library (npm package for JavaScript / Typescript).

This is not yet implemented in the project. There are no interfaces defining the messages.

Logger as a dependency

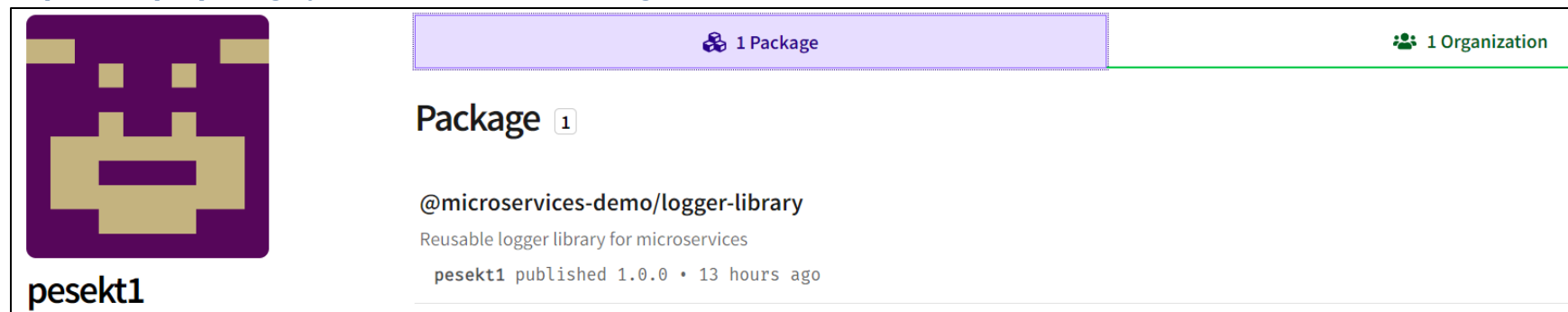
We want to have logging functionality in each microservice. Because they are all written in the same language (TypeScript), we don't want to repeat the same logic multiple times. Instead, we can create our own library and publish it as npm package which each microservice can import and use. For other languages we have similar possibilities as discussed above.

NOTE: We are hosting the library source code in our monorepo for the simplicity. But it could have its own project and repository.

The antipattern would be if bank and fraud-detection were importing from the common code – here in dependencies folder.


```
✓ MICROSERVICES-LOAD-BALANCING
> .github
> bank
> dependencies
> docker-compose-config-yaml
> fraud-detection
```

To publish npm package, you need an account and organization:



The screenshot shows the npm profile page for the user 'pesekt1'. On the left is a pixelated avatar. To the right, a purple bar indicates '1 Package' and a green bar indicates '1 Organization'. Below this, the package '@microservices-demo/logger-library' is listed with the description 'Reusable logger library for microservices'. It shows the package was published by 'pesekt1' as version '1.0.0' '13 hours ago'.

Remember to also use .npmignore to ignore things like node_modules, dist, etc.

In package.json, we need to specify name and also organization scope:

```
"name": "@microservices-demo/logger-library",
```

This means that the name will be logger-library and it will be published under microservices-demo organization which we of course need to create before we publish.

Publish the package:

```
npm run build
npm login
```

```
npm publish --access public
```

Under our account, we can verify that our organization has this package:

microservices-demo

Package 1

@microservices-demo/logger-library

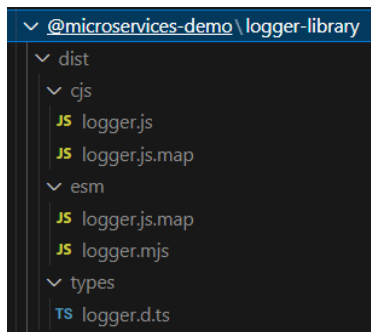
Reusable logger library for microservices

pesekt1 published 1.0.0 • 13 hours ago

We can set it public or private – in real world, we would typically use private package only for our company.

The library is configured to support both ESM and CommonJS. It supports both require and import.

We can see it when we install the library and look in the node_modules:



In the library's package.json, we configure it like this:

```
"main": "dist/cjs/logger.js",  
"module": "dist/esm/logger.mjs",
```

```
"types": "dist/types/logger.d.ts",
"files": [
  "dist"
],
"exports": {
  ".": {
    "import": "./dist/esm/logger.mjs",
    "require": "./dist/cjs/logger.js"
  }
},
```

NOTE: There are also 3 sets of tsconfig files – base, cjs, and esm.

Using the package in our applications:

```
npm install @microservices-demo/logger-library
```

```
import createLogger from "@microservices-demo/logger-library";
const logMessage = createLogger("transaction-receiver");
```

```
logMessage("Connected to RabbitMQ and queue asserted!", {
  level: "info",
});
```

NOTE: The service name is automatically included because we initialize it when calling createLogger().

NOTE: Github actions can give problems when running the workflows, you need to make sure that your package.json and tsconfig are correctly configured.

RabbitMQ

Documentation: <https://www.rabbitmq.com/>

RabbitMQ is a messaging and streaming broker. RabbitMQ supports several open standard protocols, including AMQP and MQTT.

Common use cases for RabbitMQ in microservices architectures:

- **Asynchronous Processing:** Microservices can publish messages to a queue, allowing other services to process them later without blocking execution. Or if a consumer is down, messages will be waiting in the queue for later processing. **With HTTP protocol, there would be an error and the message would be lost.**
- **Load Balancing:** Multiple instances of a microservice can consume messages from a queue (competing for the messages). We can scale horizontally (for example using Kubernetes).
- **Event-Driven Communication:** Services can publish events (e.g., "Order Created"), which multiple subscribers can react to without direct dependencies.
- ...

We are using official image – see the docker-compose yaml file. It provides various exchanges on which we can run queues.

Docker-compose example:

```
rabbitmq:
  image: rabbitmq:3-management
  container_name: rabbitmq
  ports:
    - "5672:5672" # RabbitMQ protocol port
    - "15672:15672" # RabbitMQ management console
  environment:
    RABBITMQ_DEFAULT_USER: guest
```

```
RABBITMQ_DEFAULT_PASS: guest
networks:
  - rabbitmq-net
```

Example of connection to RabbitMQ and publishing a message:

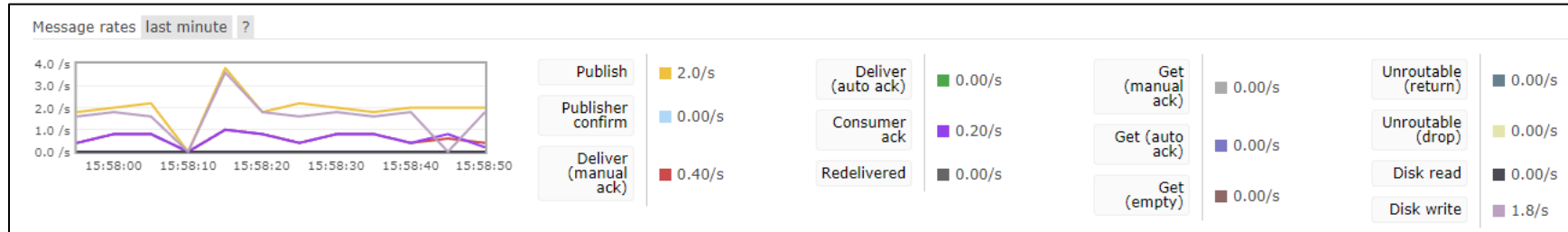
```
import amqplib from "amqplib";
const RABBITMQ_URL = process.env.RABBITMQ_URL as string;
const QUEUE_NAME = process.env.QUEUE_NAME as string;
let channel: amqplib.Channel;

export async function initializeMessageQueue() {
  try {
    const connection = await amqplib.connect(RABBITMQ_URL);
    channel = await connection.createChannel();
    await channel.assertQueue(QUEUE_NAME);
    logMessage("Connected to RabbitMQ and queue asserted!", {
      level: "info",
    });
  } catch (error) {
    logMessage(`RabbitMQ error: ${error as Error}.message`, {
      level: "error",
    });
  }
}

// Publish a message to queue:
channel.sendToQueue(QUEUE_NAME, Buffer.from(JSON.stringify(transaction)));
```

RabbitMQ Management GUI

It is part of RabbitMQ, by default running at <http://localhost:15672/>



Queues

Overview				
Virtual host	Name	Type	Features	State
/	acceptedTransactions	classic	D	running
/	suspiciousTransactions	classic	D	running
/	transactionPayloads	classic	D	running

Exchanges

Virtual host	Name	Type	Features	Message rate in	Message rate out
/	(AMQP default)	direct	D	1.2/s	1.4/s
/	amq.direct	direct	D		
/	amq.fanout	fanout	D		
/	amq.headers	headers	D		
/	amq.match	headers	D		
/	amq.rabbitmq.trace	topic	D I		
/	amq.topic	topic	D		

RabbitMQ Exporter

- Purpose: Exposes RabbitMQ metrics in a format that can be scraped by Prometheus.
- Key Features:
 - Exports metrics like queue length, message rates, and connection stats.
 - Makes it possible to monitor RabbitMQ and use these metrics for autoscaling.
- Setup:
 - Deploy RabbitMQ Exporter as a sidecar container alongside RabbitMQ or as a separate service.
 - Configure it to connect to your RabbitMQ server.

NOTE: Tools like Prometheus can now scrape the metrics directly from RabbitMQ without the need for the exporter.

In our demo, we run it in Kubernetes – code snippet from the deployment yaml:

```
spec:
  containers:
  - name: rabbitmq-exporter
    image: kbudde/rabbitmq-exporter
```

Docker Desktop

Docker Desktop is an application that provides a user-friendly interface and integrated development environment for working with **Docker** containers on Windows and macOS. It includes the **Docker Engine**, **CLI tools**, **Kubernetes**, and additional features to simplify containerized application development.

Key Features

- **Docker Engine** – Runs and manages containers on your local machine.
- **Docker CLI & GUI** – Allows you to interact with Docker via command-line tools and a graphical interface.
- **Kubernetes Support** – Provides an optional single-node Kubernetes cluster for local development.
- **Volume & Networking Management** – Helps manage persistent storage and networking for containers.
- **Docker Compose** – Enables defining and running multi-container applications using a YAML file.
- **Automatic Updates & Security** – Keeps Docker and Kubernetes up-to-date with security patches.
- **WSL 2 Integration (Windows)** – Improves performance on Windows by running Docker in a lightweight Linux VM.
- **Resource Limits** – Allows controlling CPU and memory usage to optimize performance.

Usage Scenarios

- **Local Development** – Build, test, and run containerized applications locally before deploying to production.
- **Microservices** – Develop and test microservices architectures with isolated containers.
- **Kubernetes Testing** – Run Kubernetes locally to test workloads before deploying to a real cluster.
- **CI/CD Pipelines** – Use Docker Desktop for local testing before pushing images to a registry.

Docker compose

services:

- rabbitmq:
- mysql:
- bank:
- transaction-receiver:
- fraud-detection:
- transaction-processing:
- loki:
- promtail:
- grafana:

NOTE: We did not include the monitoring services because they are used for autoscaling which is not possible in docker-compose. Instead, we are using them in Kubernetes deployment which is explained later.

Docker Desktop:

▼	●	microservices-load-balancing	-	-	-
	●	grafana	3389cd9a5c19	grafana/grafana:10.1.0	3000:3000 ↗
	●	loki	3f86a7672b8e	grafana/loki:latest	3100:3100 ↗
	●	bank-1	e32312e778b9	pesekt1/bank:latest	
	●	fraud-detection-1	94af3a1399c8	pesekt1/fraud-detection:latest	
	●	transaction-processing-1	5e4b830403bb	pesekt1/transaction-processing:latest	
	●	transaction-receiver-1	593d5c9b8bd4	pesekt1/transaction-receiver:latest	
	●	mysql	ae9e9ee9ffa5	mysql:8.0	3307:3306 ↗
	●	promtail	070ff17b517a	grafana/promtail:2.9.1	
	●	rabbitmq	6eb087b5bac9	rabbitmq:3-management	15672:15672 ↗ 5672:5672 ↗ Show less

Commands:

```
docker-compose up --build -d
docker-compose down
```

Rebuild and run a single service:

```
docker compose up transaction-receiver --build -d
```

Waiting for other services

Microservices are dependent on rabbitMQ and consumer is also dependent on MySQL. To start the container only after the dependency container is up and running, we use [wait-for-it.sh](#) script

In the Docker file:

Bank:

```
# Run the application with the wait-for-it.sh script, set the timeout to 60 seconds  
CMD ["/wait-for-it.sh", "rabbitmq:5672", "-t", "60", "--", "npm", "start"]
```

Transaction-processing:

```
# Start the application with increased timeout using the wait-for-it.sh script  
CMD ["/wait-for-it.sh", "rabbitmq:5672", "-t", "60", "--", "/wait-for-it.sh", "mysql:3306", "-t", "60", "--", "npm", "start"]
```

Alternative solutions: [chagGPT answer](#).

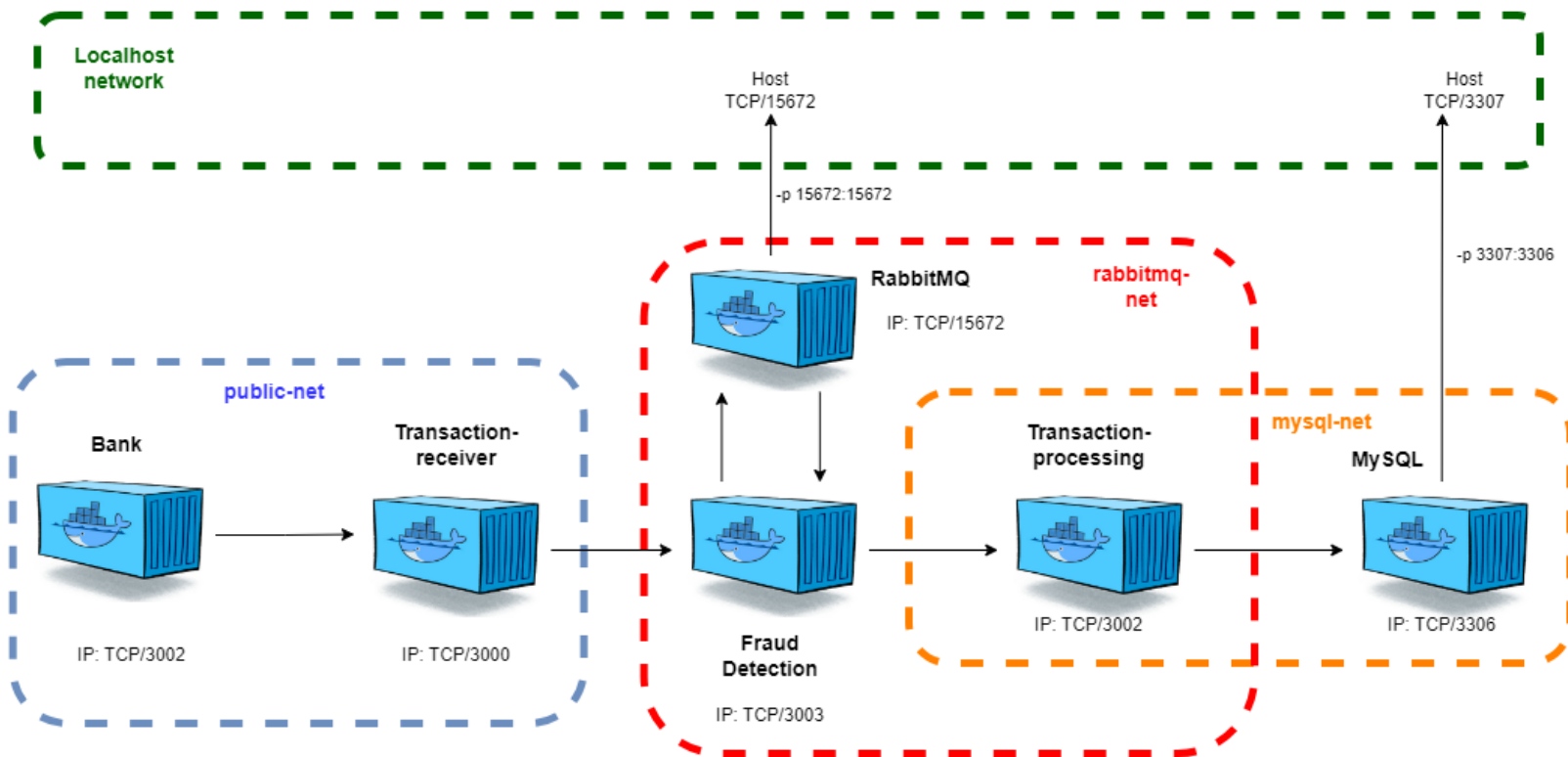
Best options:

- Healthcheck
- dockerize

Docker Compose Networks

Docker compose uses its own network within the local machine.

Docker Compose Networks



NOTE: For simplicity, we excluded the rest of the services in the schema

Scaling

We can scale the consumer microservice manually:

```
docker-compose up --scale service_name=3
```

Limitations:

- you cannot define a service name in the docker-compose file
- you cannot define a port mapping for the scaled service
- you cannot set the scale in the docker-compose file, you have to do it in the command line
- you cannot set the auto scaling rules in the docker-compose file

Mounted volumes

Mount the development folder containing the source code with the working directory on the running container. Like this the code will be automatically updated on the running container.

```
bank:
...
  volumes:
    - ./bank:/app # Mounted volume - for development purposes
```

This is not enough - we also need to run the microservice in a way that it will restart when there are changes in the code.

In package.json, we update the start script:

```
"scripts": {
  "start": "nodemon --legacy-watch --exec ts-node --transpile-only index.ts",
```

With this setup, we can have the docker-compose running while we are live coding. The applications inside the containers will restart on changes. We don't need to rebuild the images or restart the containers manually.

Kubernetes deployment

The demo has a fully automated setup using Makefile and Skaffold tools.

To start the whole system, we run:

```
make up
```

Useful info about kubernetes and how to work with it

The proper way to deploy and autoscale microservices is to use an orchestration tool like Kubernetes. Kubernetes is a container orchestration tool that automates the deployment, scaling, and management of containerized applications.

- Kubernetes needs a container registry to pull images from.
- We can do it manually (but the proper way is to set up CI/CD pipeline)

Create docker images (run from each microservice folder):

```
docker build -t pesekt1/bank-microservice .  
docker build -t pesekt1/consumer-microservice .
```

Push images to dockerhub:

```
docker push pesekt1/bank-microservice  
docker push pesekt1/consumer-microservice
```

Create Kubernetes manifest files.

- Deployment manifests: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Service manifests: <https://kubernetes.io/docs/concepts/services-networking/service/>

Set up how much CPU and Memory can each deployment use

Example (in deployment manifest yaml):

```
spec:  
  containers:
```

```

- name: transaction-receiver
  image: pesekt1/transaction-receiver:latest
  resources:
    requests:
      cpu: "100m" # Request 100m (millicpu) for the container
      memory: "128Mi" # Request 128Mi memory
    limits:
      cpu: "500m" # Limit the CPU usage to 500m
      memory: "512Mi" # Limit memory to 512Mi

```

run kubernetes manifests (stored in kubernetes folder):

```
kubectl apply -f ./kubernetes
```

Overview in Docker Desktop:

Name	Container ID	Image
 k8s_rabbitmq_rabbitmq-668f6bdd8-558w	60b870324d7b	e685a2b8c9fa
 k8s_mysql_mysql-579d99976d-89rgn_de	34f7dd5c3e8f	3818a28b4a67
 k8s_rabbitmq-exporter_rabbitmq-exporte	a6c73cb92c6b	kbudde/rabbitmq-exporter
 k8s_prometheus_prometheus-57ddc8fd5	0b73821e1d41	prom/prometheus
 k8s_consumer-microservice_consumer-n	bc3cba68e91c	pesekt1/consumer-microservice
 k8s_bank-microservice_bank-microservic	e2ae87fdd010	pesekt1/bank-microservice

Verify deployments:

```
kubectl get deployments
kubectl get pods
kubectl get services
```

Delete deployments (stored in kubernetes folder):

```
kubectl delete -f ./kubernetes
```

MySQL in Kubernetes

You can access it via MySQL Workbench because we exposed NodePort in the service manifest yaml.

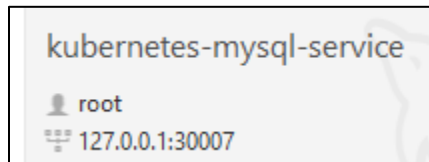
```
ports:
  - port: 3306
    targetPort: 3306
    nodePort: 30007 # NodePort for MySQL
```

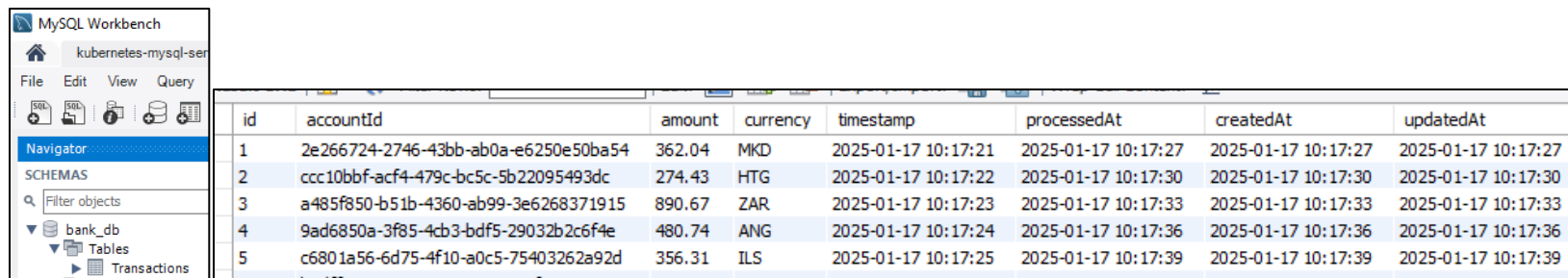
Credentials are set in the deployment manifest yaml:

```
env:
  - name: MYSQL_ROOT_PASSWORD
    value: "root"
```

- user: root
- password: root

Connection in MySQL Workbench:





The screenshot shows the MySQL Workbench interface. On the left, the 'Navigator' pane displays the 'bank_db' schema with 'Tables' and 'Transactions' listed. The main window shows a table with the following data:

id	accountId	amount	currency	timestamp	processedAt	createdAt	updatedAt
1	2e266724-2746-43bb-ab0a-e6250e50ba54	362.04	MKD	2025-01-17 10:17:21	2025-01-17 10:17:27	2025-01-17 10:17:27	2025-01-17 10:17:27
2	ccc10bbf-acf4-479c-bc5c-5b22095493dc	274.43	HTG	2025-01-17 10:17:22	2025-01-17 10:17:30	2025-01-17 10:17:30	2025-01-17 10:17:30
3	a485f850-b51b-4360-ab99-3e6268371915	890.67	ZAR	2025-01-17 10:17:23	2025-01-17 10:17:33	2025-01-17 10:17:33	2025-01-17 10:17:33
4	9ad6850a-3f85-4cb3-bdf5-29032b2c6f4e	480.74	ANG	2025-01-17 10:17:24	2025-01-17 10:17:36	2025-01-17 10:17:36	2025-01-17 10:17:36
5	c6801a56-6d75-4f10-a0c5-75403262a92d	356.31	ILS	2025-01-17 10:17:25	2025-01-17 10:17:39	2025-01-17 10:17:39	2025-01-17 10:17:39

RabbitMQ in Kubernetes

You can access RabbitMQ Manager GUI on: <http://localhost:30008/> because we exposed NodePort in the service yaml.

```
- name: management
  protocol: TCP
  port: 15672
  targetPort: 15672
  nodePort: 30008 # NodePort for management UI
```

RabbitMQ Exporter in Kubernetes

Port forward RabbitMQ Exporter to access it on localhost in the browser:

```
kubectl port-forward svc/rabbitmq-exporter 9419:9419
```

- Open RabbitMQ Exporter in your browser: <http://localhost:9419>.
- You can go to metrics: <http://localhost:9419/metrics>


```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0.000134109
go_gc_duration_seconds{quantile="0.25"} 0.000199312
go_gc_duration_seconds{quantile="0.5"} 0.000222313
go_gc_duration_seconds{quantile="0.75"} 0.000392924
```

Port forwarding

Exposing the services inside the local Kubernetes cluster to the localhost, so that we can access them for example via browser, or other tools like mysql workbench.

Example: port forward Prometheus service on localhost port 9090:

```
kubectl port-forward svc/prometheus 9090:9090
```

Sometimes port forwarding is hanging, and the port is blocked which prevents from using it again. To kill the existing process, you can use these commands (example for port 9414):

```
netstat -ano | findstr :9419
```

```
C:\Users\Tomas>netstat -ano | findstr :9419
TCP    127.0.0.1:9419      0.0.0.0:0          LISTENING        26936
TCP    [::1]:9419         [::]:0             LISTENING        26936
```

```
taskkill /PID 26936 /F
```

Alternative to port forwarding:

We can set the service yaml to type [NodePort](#) and add a nodePort property. Now this service is available on <http://localhost:30000>

```
spec:
  type: NodePort
  ports:
    - port: 9090
      targetPort: 9090
      nodePort: 30000
  selector:
    app: prometheus
```

- In a single-node local Kubernetes cluster (like Docker Desktop Kubernetes), the "node" is your own machine.
- Since NodePort exposes the service on a port on every node, and there's only one node (your machine), the service is accessible on localhost:<NodePort>.
- In a real multi-node Kubernetes cluster, NodePort exposes the service on each node's IP, and you must connect using the node's external IP (http://<NodeIP>:<NodePort>), not localhost.

Scaling with Kubernetes

Scaling in Kubernetes is very powerful feature: <https://kubernetes.io/docs/concepts/workloads/autoscaling/>

Types of scaling:

- [Horizontal scaling](#) – adding more replicas
- [Vertical scaling](#) – adding more resources

How to trigger scaling:

- Manual Scaling
- Autoscaling

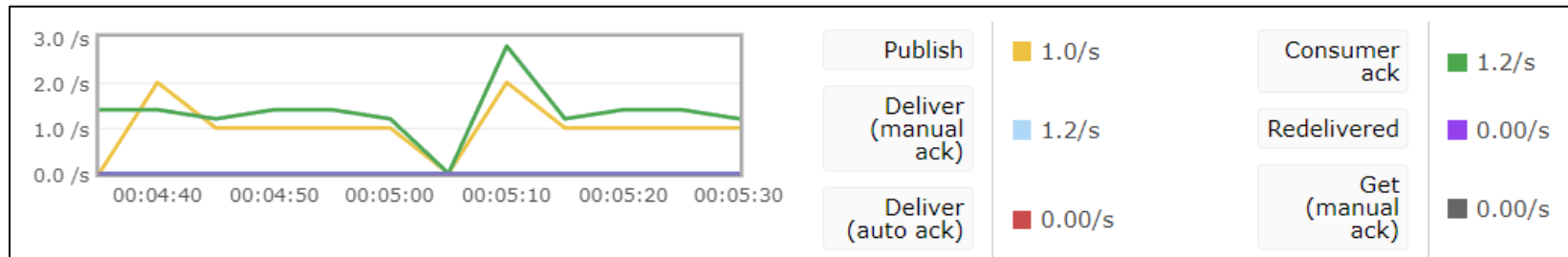
Manual scaling

We can scale manually with this command:

```
kubectl scale deployment consumer-microservice --replicas=4
```

- k8s_consumer-microservice_consumer-n eba03ecc3fa8
- k8s_consumer-microservice_consumer-n 5b20a8d807c1
- k8s_consumer-microservice_consumer-n f9b612616942
- k8s_consumer-microservice_consumer-n bc3cba68e91c

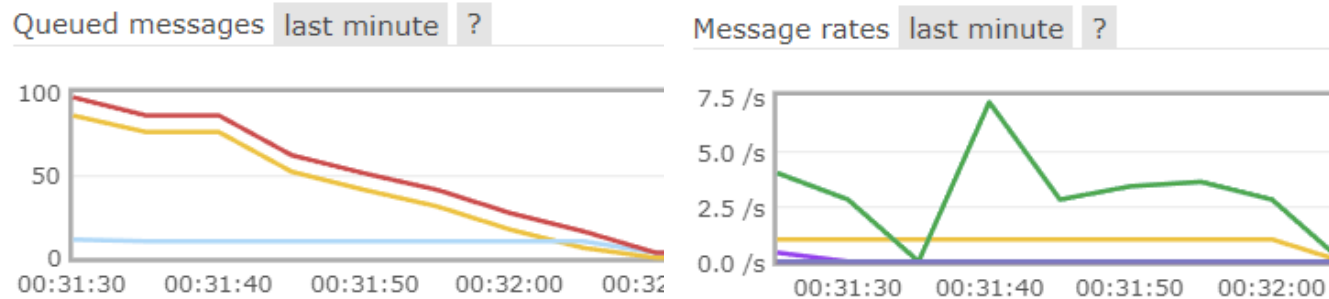
Now we can see in RabbitMQ that more messages are being consumed:



If we want to quickly consume the accumulated messages in the queue, we need many replicas:

```
kubectl scale deployment consumer-microservice --replicas=15
```

Now we can see that more messages are being consumed per second:



Autoscaling

- For example, we can use HPA – horizontal pod autoscaler: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- We can also use external services like KEDA, which is explained in the next section.

KEDA

Install and deploy KEDA: <https://keda.sh/docs/2.16/>

You can install and use KEDA by applying the KEDA manifests directly using kubectl. Here are the steps to install KEDA and configure it to use Prometheus metrics for scaling your deployment.

```
kubectl apply --server-side -f https://github.com/kedacore/keda/releases/download/v2.16.1/keda-2.16.1.yaml
```

We get 3 containers running:

●	k8s_keda-admission-webhooks_keda-adr 926daada9ea4	kedacore/keda-admission-webhooks
●	k8s_keda-metrics-apiserver_keda-metrics 410b090d0495 	kedacore/keda-metrics-apiserver
●	k8s_keda-operator_keda-operator-76dd97 0a578fcf1076	kedacore/keda

KEDA Scale object

There are many scalers that can be used: <https://keda.sh/docs/2.16/scalers/>

Scaling based on RabbitMQ metrics

We can use metrics from Prometheus – described in section Monitoring

Monitoring is the process of continuously collecting, processing, and analyzing metrics from different parts of an application or infrastructure. The purpose is to detect issues, optimize performance, and ensure system stability.

Types of Monitoring:

- Infrastructure Monitoring: Tracks CPU, memory, disk usage, network traffic.
- Application Performance Monitoring (APM): Observes response times, error rates, and request volumes.
- Service Monitoring: Checks availability, health, and interactions between microservices.

Prometheus.

Transaction-processing-scaledobject.yml: Sets up the scaling of a microservice using Prometheus metrics:

(the same was implemented also for the fraud-detection microservice)

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: transaction-processing-scaledobject
  namespace: default
```

```
spec:
  scaleTargetRef:
    name: transaction-processing
  minReplicaCount: 1
  maxReplicaCount: 10
  pollingInterval: 30 # Polling interval in seconds
  cooldownPeriod: 60 # Cooldown period in seconds
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus.default.svc.cluster.local:9090
      metricName: rabbitmq_queue_messages
      query: avg(rabbitmq_queue_messages{queue="acceptedTransactions"})
      threshold: '20'
```

NOTE: KEDA uses a default polling interval 30sec and cooldown period 60sec causing a delay before it starts scaling up or down.

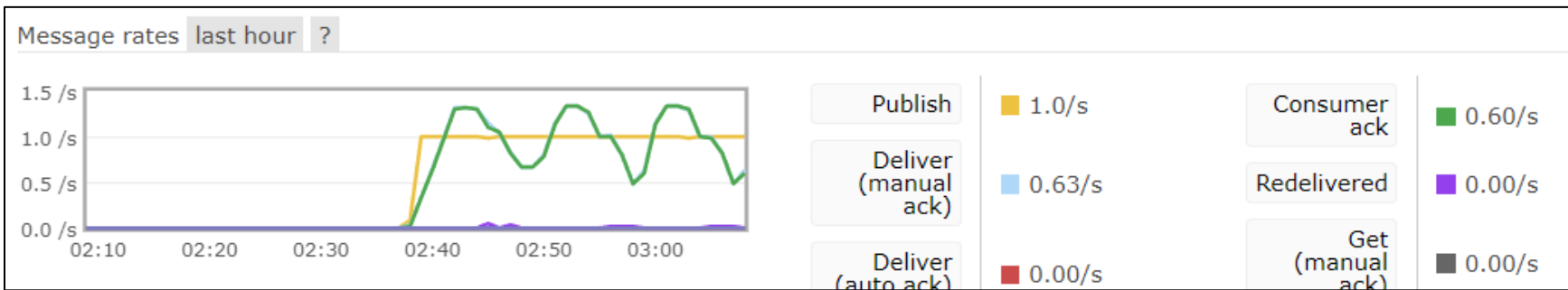
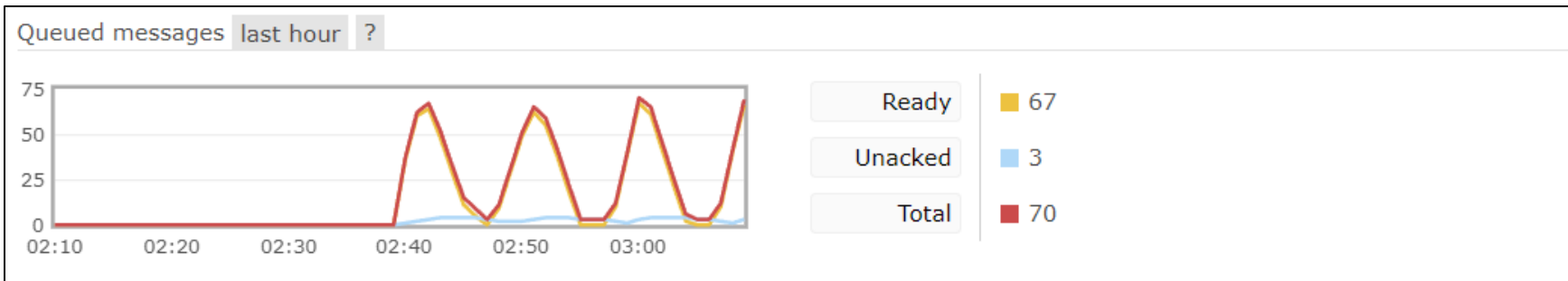
KEDA automatically deploys the HPA (Horizontal Port Autoscaler) based on scaling object yaml:

```
kubectl get hpa
```

```
$ kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
keda-hpa-cpu-scaledobject	Deployment/transaction-receiver	cpu: 2%/1%	1	3	3
keda-hpa-transaction-processing-scaledobject	Deployment/transaction-processing	18250m/20 (avg)	1	10	4

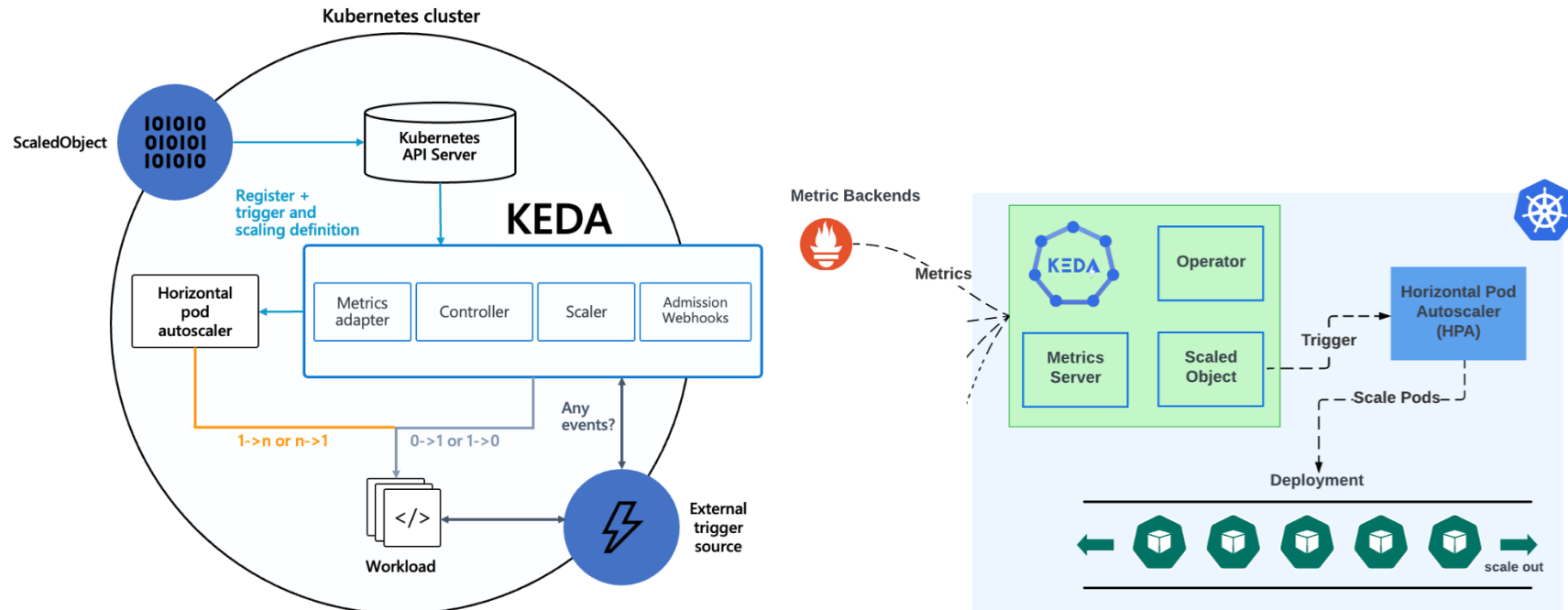
- Kubernetes HPA: <https://www.kubecost.com/kubernetes-autoscaling/kubernetes-hpa/>
- Now the consumer microservice is scaled based on the rules defined in the scaleobject.yaml.
- We can observe the results in RabbitMQ – we can see the number of consumers, consumed messages, messages in the queue, etc.



Scaled replicas:

●	k8s_transaction-processing_tran: 58e8f92a1e89	pesekt1/transaction-processing
●	k8s_transaction-processing_tran: 429f6c3c0a54	pesekt1/transaction-processing
●	k8s_transaction-processing_tran: d155f9d661a6	pesekt1/transaction-processing

KEDA Architecture



Scaling microservices running within Kubernetes cluster based on the metrics scraped from Prometheus.

Scaling based on CPU of containers

<https://keda.sh/docs/2.16/scalers/cpu/#example>

transaction-receiver-scaledobject:

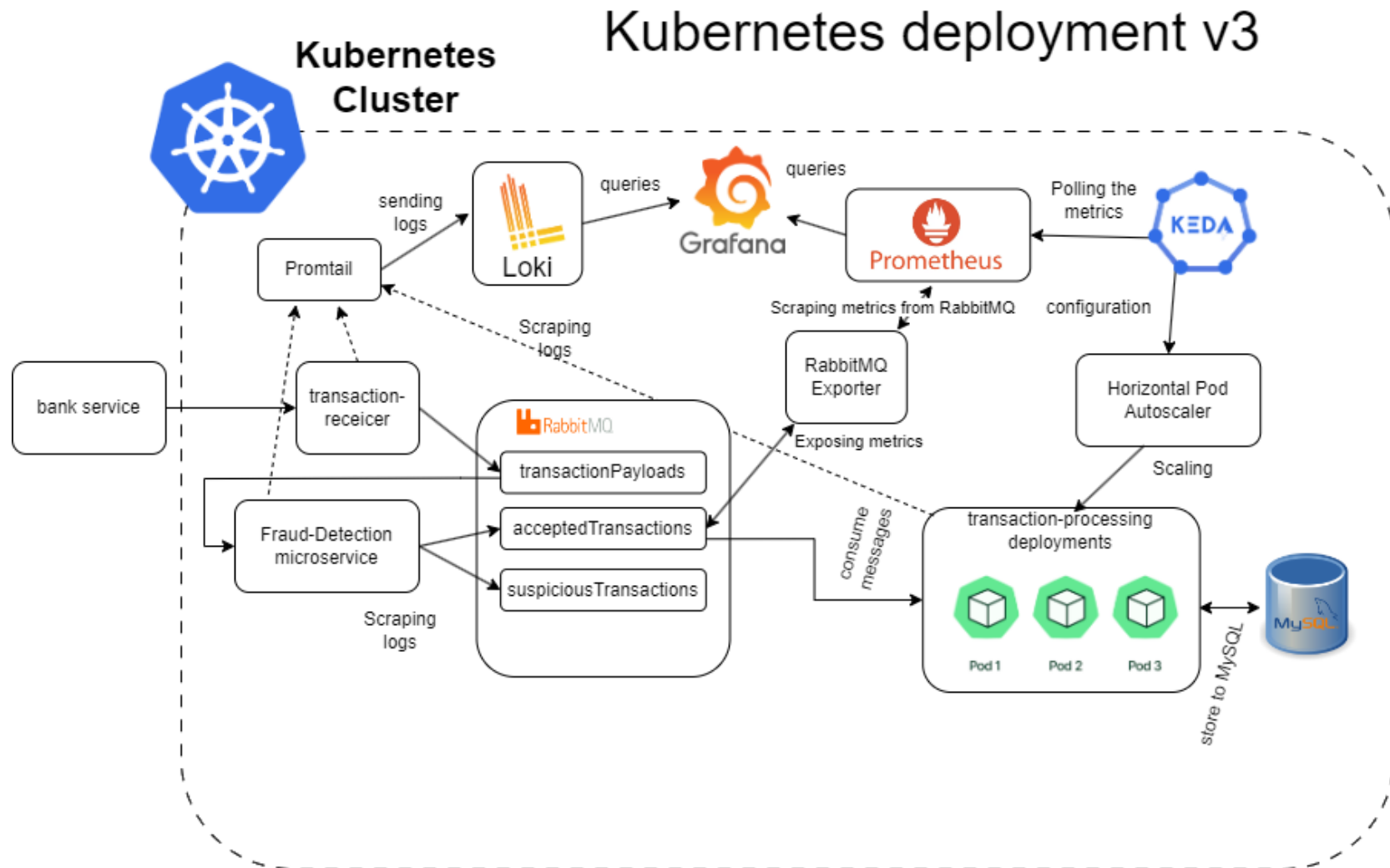
```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: cpu-scaledobject
  namespace: default
```



```
spec:
  scaleTargetRef:
    name: transaction-receiver
  minReplicaCount: 1
  maxReplicaCount: 3
  triggers:
  - type: cpu
    metricType: Utilization # Allowed types are 'Utilization' or 'AverageValue'
    metadata:
      value: "1" # Target average CPU utilization (percentage) for the deployment
```

For this to work, we need to have metrics server running – explained in a separate chapter - Metrics server.

Final solution architecture



- Kubernetes pulls the docker images from Docker hub, it does not use local images.
- Transaction-receiver and fraud-detection are also scaled with KEDA HPA object (just not shown in the schema for simplicity)

Rolling updates

If there are multiple replicas of the consumer microservices running and we update the docker image on Dockerhub, Kubernetes will initiate rolling updates which means that it will exchange one replica pod at a time.

For the local demo, it is difficult to simulate the full rolling updates process because we would need github action to communicate with our local Kubernetes cluster. This would be more fitting for a real cloud deployment for example in Azure with AKS.

That is why our CI-CD pipeline will only be Continuous Integration / Continuous Delivery – we will not really trigger the actual deployments of the new version, but we will update the Docker image of the updated microservice on Dockerhub. When Kubernetes starts a new replica, it will use this new version.

We need to set it up in Kubernetes deployment file for the microservice:

```
containers:
- name: consumer-microservice
  image: pesekt1/consumer-microservice:latest
  imagePullPolicy: Always # Ensures the latest image is always pulled
```

Start Kubernetes deployments with shell script

We create a shell script to put together all the commands and functionality we need for the Kubernetes deployment:

- Kubectl apply
- Kubectl delete - with a flag --delete
- port forwarding
- cancelling the port forwarding on delete
- installing dependencies if they are not present – in our case the KEDA and Grafana
- opening Grafana, Prometheus, and RabbitMQ Manager in the browser,
- etc.

Now we can execute the shell script in bash terminal:

Start all deployments:

```
bash kubernetes-deployment.sh
```

Delete all deployments:

```
bash kubernetes-deployment.sh --delete
```

Lens

<https://k8slens.dev/>

Lens is a Kubernetes platform that provides GUI for interaction with Kubernetes clusters, and an environment for secure and effective work within teams and organizations.

Services overview:

☐	Name	 Namespace	Type	Cluster IP	Ports	External IP	Selector	Age	Status
☐	bank-microservice	default	ClusterIP	10.105.209.4	3000/TCP	-	app=bank-n	5m33s	Active
☐	consumer-microservice	default	ClusterIP	10.98.169.141	3001/TCP	-	app=consur	5m32s	Active
☐	kubernetes	default	ClusterIP	10.96.0.1	443:6443/TCP	-		22m	Active
☐	mysql	default	NodePort	10.100.153.5	3306:30007/TCP	-	app=mysql	5m32s	Active
☐	prometheus	default	NodePort	10.106.41.60	9090:30000/TCP	-	app=prome	5m32s	Active
☐	rabbitmq	default	NodePort	10.106.231.12	5672:30009/TCP,	-	app=rabbitr	5m32s	Active
☐	rabbitmq-exporter	default	ClusterIP	10.110.50.40	9419/TCP	-	app=rabbitr	5m32s	Active

Endpoints:

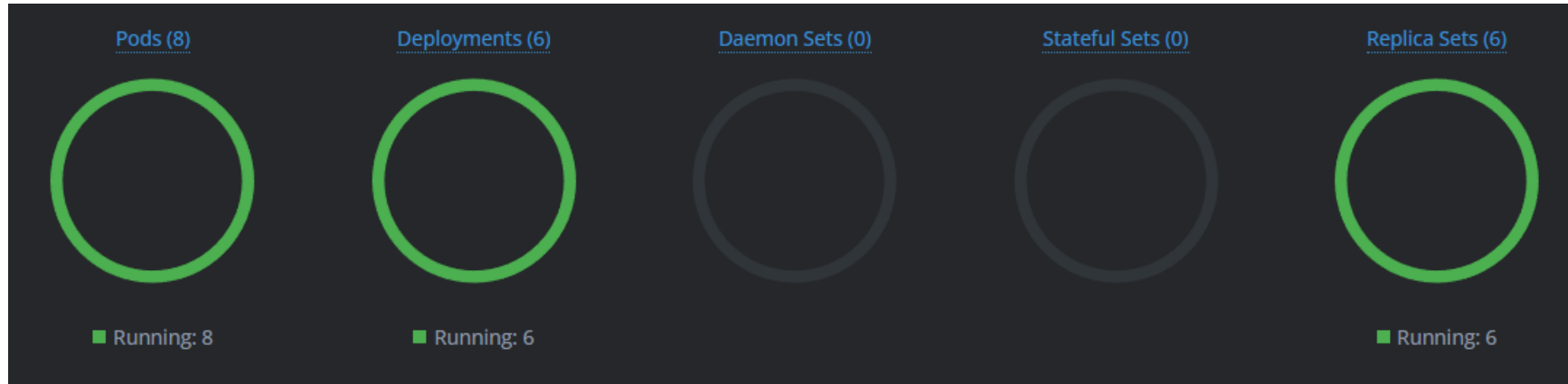
☐	Name	 Namespace	Endpoints
☐	bank-microservice	default	10.1.4.189:3000
☐	consumer-microservice	default	10.1.4.190:3001, 10.1.4.195:3001, 10.1.4.196:3001
☐	kubernetes	default	192.168.65.3:6443
☐	mysql	default	10.1.4.194:3306
☐	prometheus	default	10.1.4.191:9090
☐	rabbitmq	default	10.1.4.192:5672, 10.1.4.192:15672
☐	rabbitmq-exporter	default	10.1.4.193:9419

Notice that consumer-microservice is scaled to 3 instances, so that there are 3 endpoints.

Workloads

▼ Workloads
Overview
Pods
Deployments
Daemon Sets
Stateful Sets
Replica Sets
Replication Controllers
Jobs
Cron Jobs

Overview

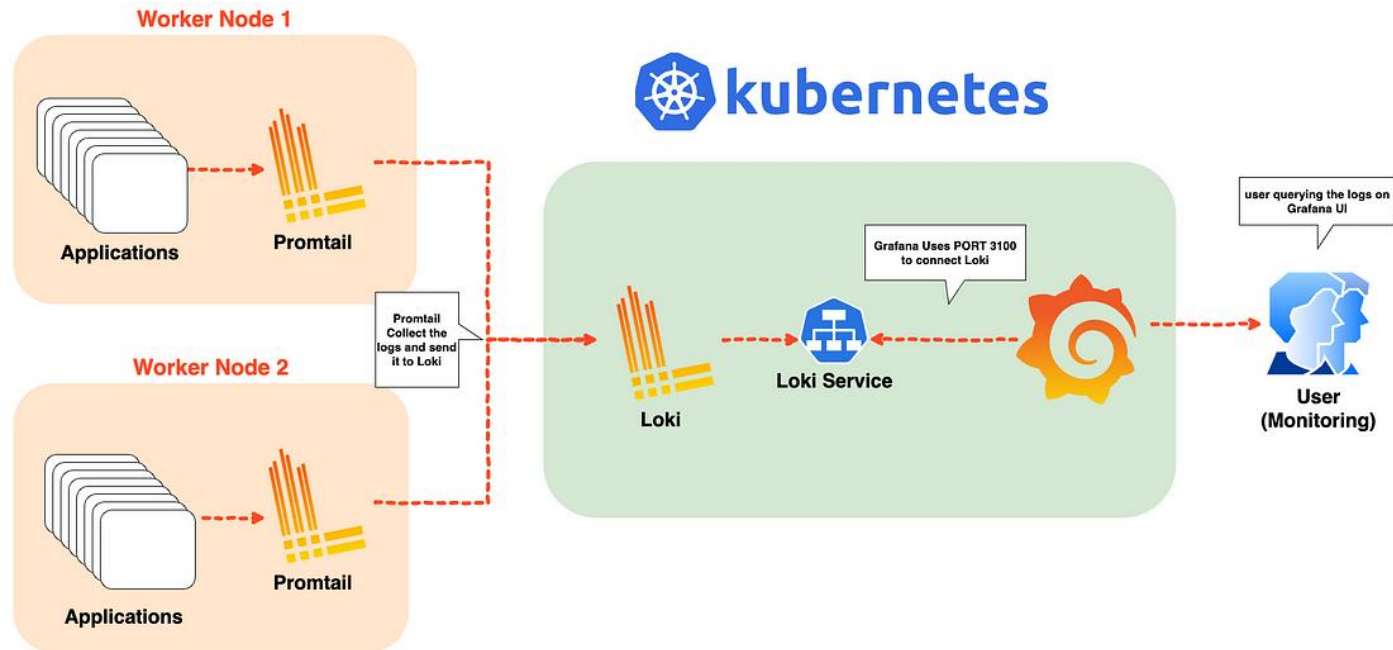


Logging

Logging is especially important for large, distributed IT systems. Logging is the process of collecting and storing logs (text-based records of events happening in an application) for the purpose of troubleshooting errors, auditing transactions, and understanding system behavior.

Tools: Grafana Loki, Grafana Alloy, Promtail, Elasticsearch, Fluentd, Jæger.

Our solution: Promtail → Loki → Grafana



We can scrape logs from different microservices – here running in the containers inside Kubernetes Cluster. NOTE: In our solution, we only have a single node because we are using local Kubernetes Cluster.

Grafana Loki

- <https://grafana.com/docs/loki/latest/>
- <https://grafana.com/oss/loki/>

Loki API: <https://grafana.com/docs/loki/latest/reference/loki-http-api/>

Note: There are different deployments. The easiest one is the monolithic deployment.

- <https://grafana.com/docs/loki/latest/get-started/deployment-modes/#monolithic-mode>

docker-compose:

```
loki:
  image: grafana/loki:latest
  container_name: loki
  ports:
    - "3100:3100"
  volumes:
    - ./docker-compose-config-yaml/loki-config.yaml:/etc/loki/local-config.yaml
    - ./loki-data:/loki
  environment:
    - LOKI_CONFIG_FILE=/etc/loki/local-config.yaml
  command: -config.file=/etc/loki/local-config.yaml
  restart: unless-stopped
```

We need loki-config.yaml in the folder docker-compose-config-yaml because we use it as mounted volume.

Examples of configuration: <https://grafana.com/docs/loki/latest/configure/examples/configuration-examples/#configuration>

When we map the port to the host, we can see the metrics here: <http://localhost:3100/metrics>

Test:

```
curl http://localhost:3100/loki/api/v1/labels
```

```
C:\Users\Tomas>curl http://localhost:3100/loki/api/v1/labels
{"status":"success","data":["filename","job","service_name"]}
```

LogQL query language: https://grafana.com/docs/loki/latest/query/log_queries/

Examples: (in Grafana dashboard with Loki data source)

```
{job="transaction-receiver"} # all logs for this service
{job="transaction-receiver"} |~ "info" # only info level
{job="transaction-receiver"}[5m] # last 5min
{job="transaction-receiver", user_id="1234"} # logs where user_id = 1234
count_over_time({job="transaction-receiver"}[1m]) # count of logs over 1min
{job="transaction-receiver"} | json | message=~".*Simulated.*" # logs contain the word Simulated
{job="transaction-receiver"} | json | status="FAIL" | level="error" # multiple conditions
```

Object Storage for Logs

Loki stores logs as compressed files in object storage systems. Examples include:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage
- MinIO
- Filesystem (for local testing or small setups) – our case.

Promtail

NOTE: Grafana suggest using Grafana Alloy instead of Promtail.

<https://grafana.com/docs/loki/latest/send-data/promtail/>

Docker-compose:

```
promtail:
  image: grafana/promtail:2.9.1
  container_name: promtail
  volumes:
    - ./logs:/var/log
```

```
- ./docker-compose-config-yaml/promtail-config.yaml:/etc/promtail/config.yaml
command: -config.file=/etc/promtail/config.yaml
```

We need promtail-config.yaml in the folder docker-compose-config-yaml because we use it as mounted volume.

Now we need to bind volumes to the microservices – so that promtail can read from there:

```
transaction-receiver:
...
  volumes:
    - ./logs:/var/log
```

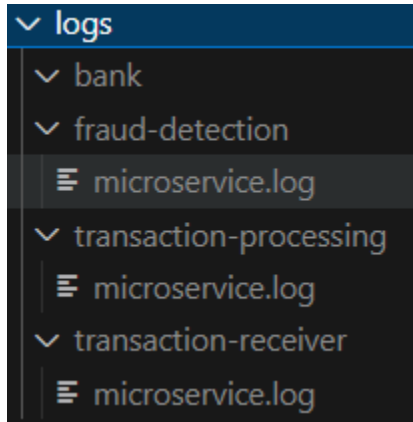
We set the scraping in promtail-config:

```
- job_name: microservices
  static_configs:
    - targets:
        - promtail:9080
      labels:
        job: transaction-receiver
        __path__: /var/log/transaction-receiver/*.log
```

We also configure the integration with Loki:

```
# Integration with Grafana Loki:
clients:
  - url: http://loki:3100/loki/api/v1/push
```

We get a log folder with subfolders with log files for each microservice from where Promtail is scraping the logs



NOTE: Alternatively, we could scrape logs directly from the container console.log which would be simpler and would not require the file storage of logs.

Monitoring

Monitoring is the process of continuously collecting, processing, and analyzing metrics from different parts of an application or infrastructure. The purpose is to detect issues, optimize performance, and ensure system stability.

Types of Monitoring:

- Infrastructure Monitoring: Tracks CPU, memory, disk usage, network traffic.
- Application Performance Monitoring (APM): Observes response times, error rates, and request volumes.
- Service Monitoring: Checks availability, health, and interactions between microservices.

Prometheus

- **Purpose:** Scrapes metrics from RabbitMQ Exporter and other sources to store and query them.
- **Key Features:**
 - Time-series database optimized for metrics collection.

- Allows querying metrics via PromQL (Prometheus Query Language).
- **Setup:**
 - Install Prometheus in your Kubernetes cluster (using Helm or manifests) – we will use manifests to run it.
 - Configure it – for example to scrape metrics from RabbitMQ Exporter.

Example of Prometheus-configmap configuring scraping metrics from the rabbitmq-exporter:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s

    scrape_configs:
      - job_name: 'rabbitmq'
        metrics_path: /metrics
        static_configs:
          - targets: ['rabbitmq-exporter:9419']
```

Check that Prometheus is running:

```
kubectl get pods
```

Port forward Prometheus to access it on localhost in the browser:

```
kubectl port-forward svc/prometheus 9090:9090
```

Open Prometheus in your browser: <http://localhost:9090>.

Search for RabbitMQ metrics (e.g., `rabbitmq_queue_messages_ready`)

The screenshot shows the Prometheus web interface. At the top, there's a header with the Prometheus logo and navigation icons. Below the header is a search bar containing the query `>_ rabbitmq_queue_messages` and an 'Execute' button. Under the search bar are tabs for 'Table', 'Graph', and 'Explain'. The 'Table' tab is selected. Below the tabs, there's a 'Evaluation time' field and 'Load time: 24ms' and 'Result series: 1'. The main content area displays a table with one row of data:

<code>rabbitmq_queue_messages{cluster="rabbit@rabbitmq-668f6bdd8-558wz", durable="true", instance="rabbitmq-exporter:9419", job="rabbitmq", queue="bankPackets", self="1", vhost="/"} 268</code>
--

We can also investigate the statuses:

The screenshot shows the 'Status' dropdown menu in the Prometheus web interface. The menu is open, showing the following options:

- Monitoring status
- Target health
- Rule health
- Service discovery

The screenshot shows the 'Server status' dropdown menu in the Prometheus web interface. The menu is open, showing the following options:

- Runtime & build information
- TSDB status
- Command-line flags
- Configuration
- Alertmanager discovery

Metrics server

We need metrics server to be able to measure metrics from the containers, like CPU, memory, etc. It is described in the section [Scaling based on CPU of containers](#)

- Installation is explained here: <https://dev.to/docker/enable-kubernetes-metrics-server-on-docker-desktop-5434>
- Download: <https://github.com/kubernetes-sigs/metrics-server/releases>

We just need to download the yaml file and make a small change in the deployment yaml. (described in the tutorial above). We can have separate yaml files or a single file metrics-server/components.yaml – as in our demo.

Add this to the deployment yaml:

```
spec:
  containers:
  - args:
    - --kubelet-insecure-tls # This is required to allow metrics-server to connect to kubelet
```

Test if metrics server is running:

If it is not properly running, we get an error:

```
kubectl top node
error: Metrics API not available
```

If it is running, you should get the docker-desktop node (there is a single node in Kubernetes local cluster)

```
$ kubectl top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
docker-desktop	442m	3%	5417Mi	70%

Grafana

Grafana is an open-source data visualization and monitoring platform. It allows you to create custom dashboards to visualize data from various sources such as **Prometheus**, **Elasticsearch**, **InfluxDB**, **MySQL**, etc.

Key Features

1. **Custom Dashboards:** You can design interactive and customizable dashboards to display various metrics in the form of graphs, charts, and tables.
2. **Multi-Data Source Integration:** Grafana integrates with a wide range of data sources, enabling you to pull data from different services and databases.
3. **Real-Time Monitoring:** Grafana enables real-time data monitoring, which is crucial for observing system performance and health.
4. **Alerting:** You can set up alerts that notify you when certain thresholds are reached, sending notifications via channels like email, Slack, etc.
5. **Easy-to-Use Interface:** It offers an intuitive user interface, making it easy for both developers and non-technical users to understand and visualize metrics.

Common Use Cases

- **Infrastructure Monitoring:** Visualizing system and application metrics.
- **Application Performance Monitoring (APM):** Analyzing performance data for applications.

Grafana is widely used in monitoring environments, particularly with time-series databases like Prometheus.

Use Grafana when

- You are monitoring multiple services (e.g., RabbitMQ, Kubernetes, database, etc.) and want to visualize them in one unified dashboard.
- You require more advanced metrics visualization and alerting capabilities.
- You want to store metrics long-term with Prometheus and analyse them over time.

Documentation: <https://grafana.com/docs/grafana/latest/>

Installation:

In our demo, the main shell script checks if Grafana exists, and if does not, it will install it using the premade Grafana-install.sh shell script. This installs Grafana globally in your Kubernetes cluster using Helm.

The main part of the shell script:

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

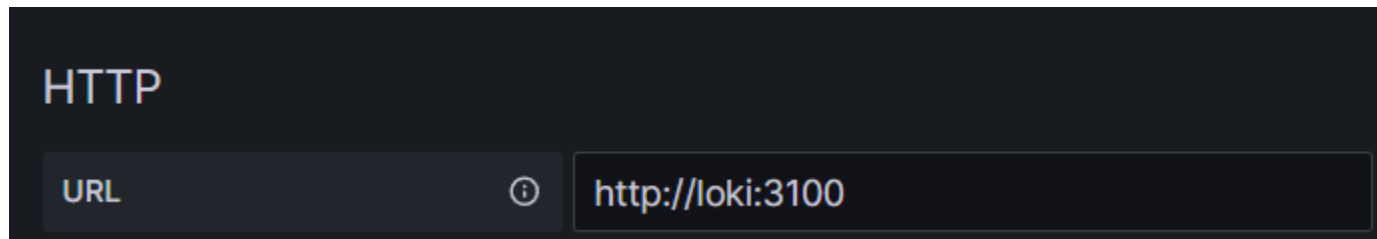
# Step 3: Install Grafana with persistence
echo "📦 Installing Grafana with persistent storage..."
helm install grafana grafana/grafana \
  --set service.type=NodePort \
  --set service.nodePort=30001 \
  --set persistence.enabled=true \
  --set persistence.size=2Gi \
  --set persistence.storageClassName=hostpath \
  --set adminPassword='admin'
```

Dashboard:

- <http://localhost:3000/> - for docker compose
- or <http://localhost:30001/> - for Kubernetes cluster

Loki data source

URL <http://loki:3100>

A screenshot of the Grafana web interface. At the top, the word "HTTP" is displayed in a large, light-colored font. Below it, there is a configuration bar for a data source. On the left, the word "URL" is shown in a light font. To its right is a small circular icon with an information symbol. Further right, the URL "http://loki:3100" is entered into a text field.

Save & test

✓ Data source successfully connected.

Explore – Run query: (remember to set up the time range)

```
{job="transaction-receiver"}
```

```
> 2025-01-26 21:14:29.250 {"level":"info","message":"Microservice started successfully.","service":"transaction-receiver","timestamp":"2025-01-26T19:44:35.519Z"}
> 2025-01-26 21:14:29.250 {"level":"info","message":"Processing a transaction...","service":"transaction-receiver","timestamp":"2025-01-26T19:44:35.522Z"}
> 2025-01-26 21:14:29.250 {"level":"info","message":"Transaction processed successfully: Result = 5","service":"transaction-receiver","timestamp":"2025-01-26T19:44:35.522Z"}
```

TODO: Create a graph with different error levels shown together with different colors.

Show only info level:

```
{job="transaction-receiver"} |~ "info"
```

Querying multiple jobs:

```
{job=~"fraud-detection|transaction-receiver"}
```

Q Search or jump

Home > Dashboards > Loki dashboard > Edit panel

Table view ☐

logs from microservices

labels	Time	Line	tsNs
{ "detected_level": "info..."	1738112361985	{"level":"info","message"...	17381
{ "detected_level": "info..."	1738112361985	{"data":{"accountId":"baf...	17381
{ "detected_level": "info..."	1738112361903	{"level":"info","message"...	17381
{ "detected_level": "info..."	1738112361151	{"level":"info","message"...	17381

Query 1 Transform 0

Data source Loki > Query options MD = auto = 1104 Interval =

▼ A (Loki)

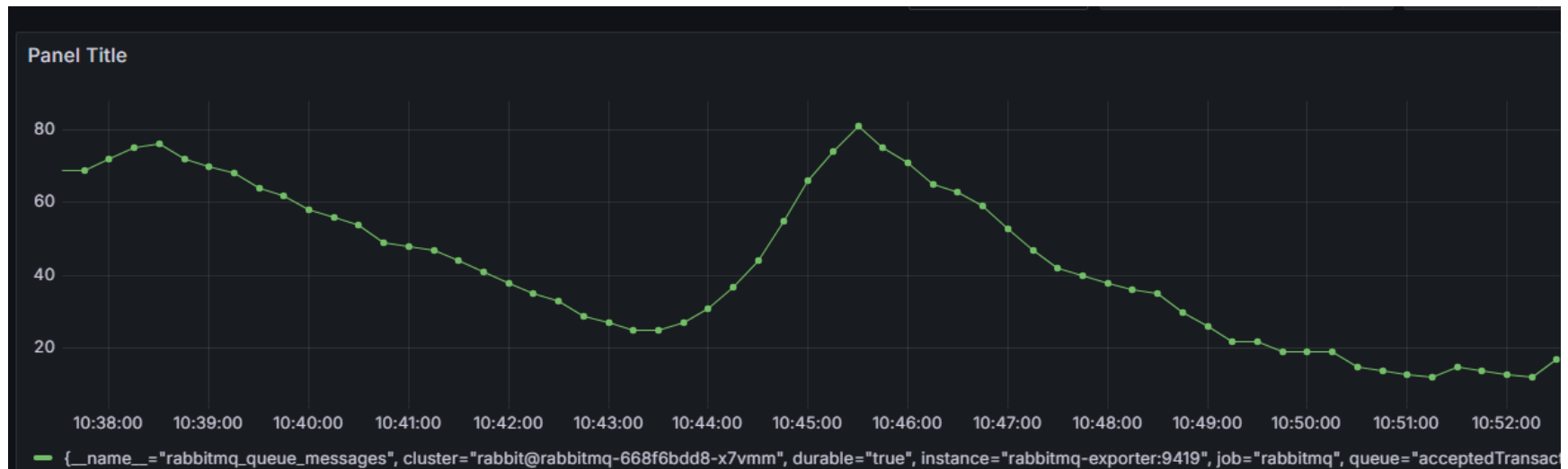
Kick start your query Label browser Explain query ☐

```
{job=~"fraud-detection|transaction-receiver"}
```

Prometheus data source

url: <http://prometheus:9090>

query example: `rabbitmq_queue_messages{queue="acceptedTransactions"}`



NOTE: RabbitMQ has also a UI showing graphs, but it has a very limited time range (1h). Prometheus on the other hand stores all the data in its time-series database.

Mysql data source

Connection

Host URL *

Database name

Authentication

Username *

Password

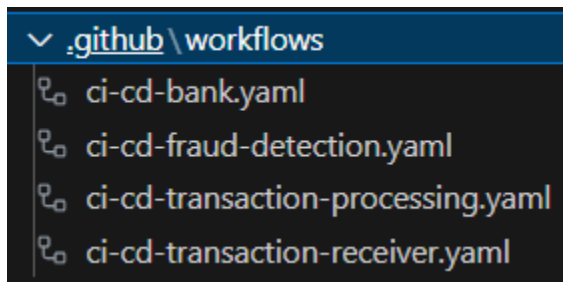
Here are some ideas what we could query in Grafana:

<https://grafana.com/docs/grafana/latest/datasources/mysql/query-editor/>

CI/CD Pipelines

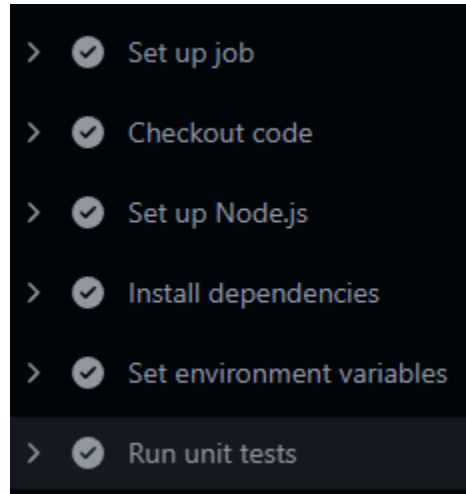
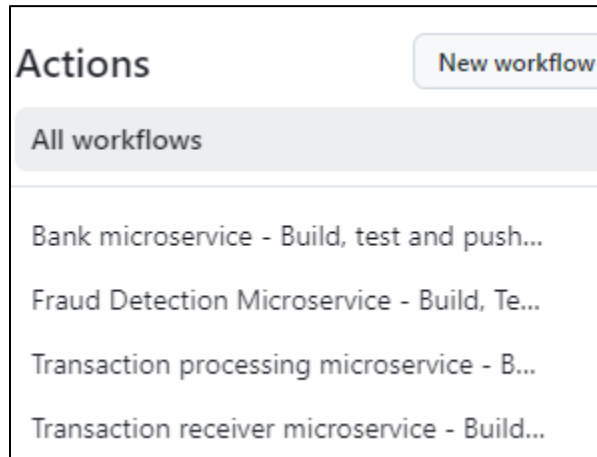
Purpose: Continuous integration – Continuous Delivery.

The result is not the actual deployment but pushing a new Docker image to Dockerhub from where Kubernetes can pull them and use them for the deployments. We don't have the actual deployment step in the pipeline because we are running local Kubernetes cluster. For the cloud deployment we could use AKS – [Azure Kubernetes Service](#).



Pipelines are also running the tests.

One workflow for each microservice:



Testing

Testing with vitest: <https://vitest.dev/>

```
npm install --save-dev vitest @types/node
```

package.json:

```
"scripts": {  
  "test": "vitest"
```

Run the test:

```
npm run test
```

NOTE: We need to update our CI-CD pipelines to run the tests.

Unit tests

For unit tests, we can mock the dependencies – for example when integrating with API, we can mock it like this:

```
const postSpy = vi.spyOn(axios, "post").mockResolvedValue({ status: 200 });
```

(from Bank microservice)

Integration tests

Testing integration with dependencies, for example: RabbitMQ and MySQL

RabbitMQ test is implemented for example in the transactions-receiver microservice.

NOTE: We need to update our CI-CD pipeline because we need the RabbitMQ dependency to run on the runner (Virtual Machine where the tests are running). For example:

```
jobs:
  build-and-push:
    runs-on: ubuntu-latest

    services:
      rabbitmq:
        image: rabbitmq:3-management
        ports:
          - 5672:5672
          - 15672:15672
```



integration-tests

26s



build-and-push

27s

System tests

Example: transactions-receiver publishes a message, we test if the message gets processed by fraud-detection and is published on one of the 2 queues.

Needs to be fixed. Works locally but not on github, so I commented it out in the workflow yaml.

End-to-end tests

Not implemented yet – there is no frontend yet.

Stress tests

Not implemented yet.

Load Balancing

Using Message Broker

When multiple replicas of the same application are consuming the messages from the message queue – in his case, they are competing (as opposed to fan-out pattern). We don't need any actual load balancer. It works automatically because the consumer replicas are consuming the messages.

HTTP Load Balancing

We need a load balancer when we have multiple instances of the API, or even when there are multiple instances of the frontend because we need a mechanism that will be redirecting the https requests to the instances. When scaling with Kubernetes, the load balancer is created for us.

Communication

REST APIs

- Swagger / Open API:
 - <https://swagger.io/>
 - Documentation: <https://swagger.io/solutions/api-documentation/>

Message Broker

- Event Catalog: <https://www.eventcatalog.dev/>
- AsyncAPI: <https://www.asyncapi.com/en>

Postman - sending messages

We can also use Postman to send messages to message broker – we need to communicate with RabbitMQ Management.

The format is `http://localhost:<rabbitmq_management_port>/api/exchanges/<vhost>/<exchange_name>/publish`

Example: (vhost is / which is encoded as `/%2F`, exchange name is `""` – empty string):

POST <http://localhost:15672/api/exchanges/%2F/publish>

Basic Auth: guest,guest:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:15672/api/exchanges/%2F//publish
- Auth Type:** Basic Auth
- Username:** guest
- Password:** guest
- Warning:** Heads up! These parameters hold sensitive data. To keep them secure, they are masked in this view.

Body:

```
{
  "properties": {},
  "routing_key": "transactionPayloads",
  "payload": "{ \"accountId\": \"550e8400-e29b-41d4-a716-446655440000\", \"amount\": \"1234.56\", \"currency\": \"USD\", \"timestamp\": \"2025-01-25T12:34:56.789Z\" }",
  "payload_encoding": "string"
}
```

Alternatively, we can always use curl:

Example: get info about all exchanges:

curl -u guest:guest <http://localhost:15672/api/exchanges/%2F>

Debugging Grafana Loki in Kubernetes cluster

- Often, we can have troubles integrating services in Kubernetes cluster. In that case, we need to apply some debugging to find out where the problem is.
- In this example, we will debug the Loki logging service.

Problem: Let us say that loki is not properly collecting the logs, so we suspect that there is something with the communication between loki and the other services.

One common technique is to deploy a testing pod and try use it to test the communication.

Start alpine image pod, so that we can test the communication with loki and open interactive shell within the pod.

```
kubect1 run -it --rm --restart=Never --image=alpine --namespace=default - sh
```

NOTE: now we have a shell terminal opened inside the pod / container. All the subsequent commands are sent to the container.

install curl in the test container:

```
apk add --no-cache curl
```

Generate actual time in bash:

```
date +%s%N
```

Test the communication:

We can find the info about the expected HTTP request in the documentation: <https://grafana.com/docs/loki/latest/reference/loki-http-api/>

Now from within the pod, send a test log to loki: (remember to replace the timestamp with the generated value)

```
curl -X POST http://loki:3100/loki/api/v1/push \ -H "Content-Type: application/json" \ -d '{  
"streams": [ { "stream": { "job": "test-job", "service_name": "test-job" }, "values": [  
["1657034190000000000", "test log line"] ] } ] }'
```

You don't get any answer if success. You need to check in Grafana, if you can see such log: (you need a dashboard with Loki data source). You can use this query:

```
{job="test-job"}
```

Panel Title		
labels	Time	Line
{ "job": "test-job", "service_name": "test-job_"	2025-01-29 22:52:08	test log line

In this example we tested that Loki is running properly and successfully receiving logs on its endpoint.